AP CYBERSECURITY | INVESTIGATION LAB - STUDENT

LAB 6

The Student Portal Data Leak

Unit 6: Securing Applications and Data | 55 to 70 minutes

Your role

You are the lead analyst after a data leak. Someone from outside the school probed the student portal in the middle of the night, and the district fears student account data is exposed. You will read a web log, **test password hashes** like an adversary would, and decide how to fix the application and the data protections.

What you will do

- Identify SQL injection, cross site scripting, and directory traversal in a web log
- Use response sizes to judge which requests probably returned data
- Show why unsalted password hashes are weak, using a small wordlist
- Compare unsalted and salted hashes
- Find a dangerous configuration file permission and plaintext credentials
- Prioritize a remediation plan

EXAM LENS

Every lab follows the same four stages: **Learn the concept, Investigate the evidence, Make a security decision, then Answer AP-style questions.** Commands show you what happened. The skill is deciding what the evidence *proves*, what it *does not prove*, and what to do next.

COMMON TRAP

This is a **fictional, controlled** investigation. Never scan, test, or access real school systems, other people's accounts, or any device you do not own or have written permission to use.

Original practice material written in the style of the AP Cybersecurity exam. Not reproduced from College Board.

Mission Briefing

All data here is fictional, including the passwords. The password-testing exercise uses a tiny wordlist on a **practice file**. You are learning why weak storage is dangerous, not breaking into anything real.

Evidence files

File	What it is
web_access.log	Portal web server log, Oct 15 (11 lines)
users.csv	User table with **unsalted SHA-256** password hashes
users_salted.csv	The same users with a salt and salted hashes
wordlist.txt	Ten very common passwords
config_listing.txt	A permission listing and contents of the portal's configuration file

Set up your VM

- Take a snapshot named `before-lab6` and copy the evidence into `~/lab6`.

```
cd ~/lab6
ls
wc -l web_access.log
```

Hash lookup (used in Task 4)

This builds a lookup table: each word in the wordlist next to its SHA-256 hash.

```
while read w; do
  h=$(printf '%s' "$w" | sha256sum | cut -d' ' -f1)
  echo "$w $h"
done < wordlist.txt > lookup.txt
cut -d, -f3 users.csv | tail -n +2 > hashes.txt
grep -F -f hashes.txt lookup.txt
```

STAGE 1 **Learn**
Be able to explain these before you touch the evidence

- ■ I can explain SQL injection, cross site scripting, and directory traversal and what each looks like in a log.
- ■ I can explain what a hash is, why it is one-way, and why identical passwords produce identical hashes.
- ■ I can explain what a salt does and why slow hashing (bcrypt, Argon2id) is better for passwords.
- ■ I can read a Linux permission string and name the safer setting.

STAGE 2

Investigate

Collect facts. Record what you observe, not what you assume.

1. Which source address is **abnormal** in `web_access.log`? How many of the 11 requests came from it, and over what time period? (Try `grep 203.0.113.61 web_access.log | wc -l`.)

2. Complete the table: for each adversary request, name the **attack** and the **evidence** in the request.

Request (shortened)	Attack?	Evidence
/portal/search?q='		
/portal/search?q=' OR '1'='1		
/portal/search?q=x' UNION SELECT username,password_hash FROM users--		
/portal/comment?text=<script>alert(1)</script>		
/portal/download?file=../../../../etc/passwd		

3. Which requests most likely **returned data**? Look at the final number in each line (response size in bytes) and the status code. Run `awk '{print $NF}' web_access.log | sort -n | tail -3`. What can you conclude and what can you **not** conclude?

4. Run the hash lookup from the setup. Which passwords were found? Which **users** have those passwords (try `grep ef92b778 users.csv` and similar)? Which user was **not** found, and why?

5. Why do mrivera and dthomas have the **same hash**? What does that tell an adversary, and what does it say about **unsalted** hashing?

6. Open `users_salted.csv`. Run `cut -d, -f4 users_salted.csv | sort | uniq -d`. What is the output, and what does it show? How do salts help? What else should a good password system use?

7. Read `config_listing.txt`. List **three problems**. Then explain why the traversal request for `db.conf` makes this especially serious.

```
-rw-rw-rw- 1 www-data www-data 412 Sep 3 10:14 db.conf

db_host=10.8.9.2
db_user=portal_admin
db_password=Sp1derWeb!2026
db_name=students
db_privileges=ALL
```

8. A teammate says: 'We will just encrypt the whole database, and SQL injection will stop being a problem.' Respond in two or three sentences.

9. Put these actions in priority order and explain your top two: force password resets for all users; fix the code (parameterized queries, input validation, file name allowlist); rotate the database password and tighten db.conf; block 203.0.113.61 and take the vulnerable pages offline; re-store passwords with salted slow hashes; add monitoring and alerts.

STAGE 3 Make a Security Decision

Turn evidence into a defensible recommendation

What can you confirm? (at least three facts, with file evidence)

What is uncertain? (at least two things the evidence does not show)

What do you recommend? Give one immediate step, one application fix, one data-protection fix, and one detection fix.

STAGE 4 AP-Style Questions

Transfer what you learned to a new situation

1. An analyst reviews a web server log entry.

```
GET /portal/search?q=x' UNION SELECT username,password_hash
FROM users--  (200, 61844 bytes)
```

Which attack does this entry indicate?

- **A.** SQL injection
- **B.** Cross site scripting
- **C.** Directory traversal
- **D.** Buffer overflow

2. A security analyst reviews a user table that was stolen from a school.

Username	Stored value (shortened)
mrivera	ef92b778
lchen	f17b44f4
dthomas	ef92b778

Which conclusion is best supported?

- **A.** The passwords are encrypted with a different key for each user
- **B.** The passwords are stored in plain text
- **C.** Two users share the same password and the hashes are not salted
- **D.** Each user's hash proves the password is strong

3. A download page accepts a file name in the URL. An adversary requests `?file=../../config/db.conf`.

Which mitigation would best prevent this type of request from succeeding?

- **A.** Encrypting all web traffic with TLS
- **B.** Validating the file parameter against an allowlist of permitted file names
- **C.** Requiring users to change passwords every 90 days
- **D.** Adding a banner that says unauthorized access is prohibited

4. A school wants to store passwords more safely after a breach.

Which approach is best?

- **A.** Store the passwords in an encrypted spreadsheet
- **B.** Store a fast unsalted hash of each password
- **C.** Store each password in plain text but restrict file permissions
- **D.** Store a unique salt and a slow password hash such as bcrypt or Argon2id for each user

Evidence Printout

Printed copies of the evidence files, for students without a terminal. The files themselves are in the evidence folder.

web_access.log

```
203.0.113.61 - [2026-10-15 03:11:02] "GET /portal/login HTTP/1.1" 200 4120
203.0.113.61 - [2026-10-15 03:12:20] "GET /portal/search?q=algebra HTTP/1.1" 200 5530
203.0.113.61 - [2026-10-15 03:13:05] "GET /portal/search?q=' HTTP/1.1" 500 311
203.0.113.61 - [2026-10-15 03:13:41] "GET /portal/search?q=' OR '1'='1 HTTP/1.1" 200 48213
203.0.113.61 - [2026-10-15 03:14:30] "GET /portal/search?q=x' UNION SELECT username,password_hash FROM
    users-- HTTP/1.1" 200 61844
203.0.113.61 - [2026-10-15 03:15:12] "GET /portal/comment?text=<script>alert(1)</script> HTTP/1.1" 200
    1980
203.0.113.61 - [2026-10-15 03:16:50] "GET /portal/download?file=../../../../etc/passwd HTTP/1.1" 404 289
203.0.113.61 - [2026-10-15 03:17:33] "GET /portal/download?file=../../config/db.conf HTTP/1.1" 200 412
10.8.4.31 - [2026-10-15 09:02:14] "GET /portal/grades?student=1001 HTTP/1.1" 200 2230
10.8.4.22 - [2026-10-15 09:05:41] "GET /portal/grades?student=1002 HTTP/1.1" 200 2218
10.8.5.14 - [2026-10-15 09:11:09] "GET /portal/grades?student=1003 HTTP/1.1" 200 2240
```

users.csv (hashes shortened by terminal wrapping)

```
id,username,password_hash
1,mrivera,ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
2,kosei,323725e8eff4df0a4974d6ea8c73017aa6467d94e09382745b7a988cec0fba0a
3,dthomas,ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
4,lchen,f17b44f4b65d227db58c3bfd1ef46781dc8f7e74612ce90af624e37e2b149c8a
5,nbell,65e84be33532fb784c48129675f9eff3a682b27168c0ea744b2cf58ee02337c5
6,apatel,78100e5081ffe6dbe764c4ea845659c2d296cd73815f8a0b8862eca3010bb62e
```

wordlist.txt

```
123456
password
password123
qwerty
letmein
Hancock1
welcome1
iloveyou
Summer2024!
abc123
```

config_listing.txt

```
$ ls -l /var/www/portal/config
-rw-rw-rw- 1 www-data www-data 412 Sep 3 10:14 db.conf

$ cat /var/www/portal/config/db.conf
db_host=10.8.9.2
db_user=portal_admin
db_password=Sp1derWeb!2026
db_name=students
db_privileges=ALL
```

KEY IDEA

Before you turn this in: every answer should name the evidence it comes from (a file name, a line, a time, or a command result). If you are not sure something is proven, write 'not confirmed' and say what evidence would confirm it.